



PL/SQL Block Structure & Variables

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

What is PL/SQL?

PL/SQL (Procedural Language/Structured Query Language) is Oracle Corporation's procedural extension to SQL. It integrates SQL queries with procedural programming constructs (loops, conditions, variables).

Core Features & Integration

- **Block Structured:** Code is structured into logically enclosed blocks. Each block is a single unit of execution.
- **Tight SQL Integration:** Supports all SQL statements, data types, and functions directly within the code.
- **Client-Server efficiency:** Groups multiple SQL queries into a single block sent to the database engine, reducing network traffic and latency.
- **Performance gains:** Compiled and run inside the Oracle Database kernel, maximizing data-processing speeds.

Benefits in Application Development

- **Procedural Logic:** Allows developers to implement complex algorithms, logical checks, and loops directly on the database side.
- **Robust Exception Handling:** Has built-in, custom error-trapping blocks to handle query failures gracefully.
- **Portability & Security:** Runs on any OS hosting Oracle Database. Business logic is secured inside database schemas.
- **Modular Architecture:** Supports packaging code into reusable procedures, functions, and packages.

PL/SQL Block Structure

PL/SQL code is written inside standardized, nested blocks containing up to three sections:

Typical Syntax Structure

```
[DECLARE]
    -- Declaration of variables, types, cursors
BEGIN
    -- Executable statements (SQL and PL/SQL)
[EXCEPTION]
    -- Error handling and trapping code
END;
/  -- Executes the block
```

Section Details

- **DECLARE (Optional):** Allocates memory for variables, constants, cursors, and user-defined exceptions. Not accessible outside the block.
- **BEGIN (Mandatory):** Contains the core procedural instructions and SQL queries. Must have at least one executable statement (can be 'NULL;').
- **EXCEPTION (Optional):** Traps errors that occur during the execution of the BEGIN section, preventing program crashes.
- **Anonymous vs. Named:** Anonymous blocks are compiled on the fly and not saved. Named blocks (procedures, functions) are stored in the database catalog.

Variables and Data Types

PL/SQL supports robust scalar, composite, reference, and LOB data types:

Scalar Types & Anchors

- **Standard SQL Scalars:** VARCHAR2, NUMBER, DATE, CHAR, BOOLEAN (unique to PL/SQL).
- **%TYPE Attribute (Anchored Type):** Declares a variable with the exact data type of a database column. Dynamically adapts to table schema changes:
`emp_salary employees.salary%TYPE;`
- **CONSTANT Declaration:** Binds a value that cannot change during block execution:
`pi CONSTANT NUMBER := 3.14159;`
- **NOT NULL:** Forces variable initialization and prevents assigning nulls.

Composite Record Types

- **PL/SQL Records:** Custom group of related fields. Similar to a struct in C/C++ or an object attribute list.
- **%ROWTYPE Attribute:** Declares a record that represents an entire row of a table or view. Fields map to table columns dynamically:
`emp_rec employees%ROWTYPE;`
-- Access: `emp_rec.first_name := 'Srinivas';`
- **Initialization:** Variables are initialized using the assignment operator `:=` or the DEFAULT keyword. Uninitialized variables default to NULL.

Exception Handling: Resilient Code

Exceptions handle runtime errors cleanly, separating error-handling from main logic:

Types of Exceptions

- **Predefined Exceptions:** Automatically raised by Oracle Database runtime for common errors:
 - **NO_DATA_FOUND:** SELECT INTO returned 0 rows.
 - **TOO_MANY_ROWS:** SELECT INTO returned >1 row.
 - **ZERO_DIVIDE:** Division by zero error.
- **User-Defined Exceptions:** Custom exceptions declared in DECLARE section and triggered manually using the RAISE statement when a business rule is violated.

Error Handling Block Syntax

EXCEPTION

WHEN NO_DATA_FOUND THEN

```
dbms_output.put_line('No Employee found!');
```

WHEN OTHERS THEN

```
-- Handles any other error
```

```
dbms_output.put_line('Error: ' || SQLERRM);
```

```
ROLLBACK;
```

Applications of PL/SQL Blocks

PL/SQL block structures are the backbone of secure database application development:

Transactional Controls

Groups complex SQL operations (e.g., deducting balance and adding to another account) into a single execution block. Commits transaction if all succeed, or performs a ROLLBACK in the EXCEPTION block if any step fails.

Batch Data Updates

Iterates through database cursors and performs batch updates. Bulk statements load millions of rows efficiently with minimal CPU overhead.

Data Validation API

Implements business validation constraints (e.g., checking user eligibility, verifying credit bounds) prior to database insertion, protecting database integrity.

ETL Processes

Extracts, transforms, and loads data during data migrations. Exception sections capture and log malformed rows to error tables, keeping migrations running.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management